UNIVERSITY INSTITUTE OF ENGINEERING

Department of Computer Science & Engineering

(BE-CSE - 6th Sem)

Subject Name: System Design

Subject Code: 23CSH-314

SUBMITTED BY:

Name: Sudhanshu Choudhary

UID: 23BCS12388

Section: 23BCS-KRG-1_A

SUBMITTED TO:

Er. Alok Kumar (E15012)

ASSIGNMENT- 1

Student Name: Sudhanshu Choudhary **UID: 23BCS12388**
Branch: BE-CSE **Section/Group: KRG-1_A**
Semester: 6th **Date of Performance: 05-02-2026**
Subject Name: System Design **Subject Code: 23CSH-314**

Q1. Explain SRP and OCP in detail with proper examples.

Ans :-

1. **Single Responsibility Principle (SRP):**

SRP states that a class should have only one reason to change. This means that a class must focus on only one responsibility or concern.
If a class performs multiple unrelated tasks, then changes in one task may unintentionally affect others, making the system difficult to maintain.

Importance:

- Easier to understand
- Better maintainability
- Easier debugging
- Lower coupling
- Higher cohesion
- Easier testing
- Improved readability

Example Violating SRP:

In the following program, a single class performs salary calculation, database storage, and report generation.

```
public class Main {

    static class Employee {
        String name;
        double baseSalary;

        Employee(String name, double baseSalary) {
            this.name = name;
            this.baseSalary = baseSalary;
        }

        double calculateSalary() {
            return baseSalary * 1.2;
        }

        void saveToDatabase() {
            System.out.println("Saving employee to database...");
        }

        void generateReport() {
            System.out.println("Generating employee report...");
        }
    }

    public static void main(String[] args) {
        Employee emp = new Employee("Rahul", 50000);
```

```
        System.out.println(emp.calculateSalary());
        emp.saveToDatabase();
        emp.generateReport();
    }
}
```

Example following SRP:

```
public class Main {

    static class Employee {
        String name;
        double baseSalary;

        Employee(String name, double baseSalary) {
            this.name = name;
            this.baseSalary = baseSalary;
        }
    }

    static class SalaryCalculator {
        double calculate(Employee e) {
            return e.baseSalary * 1.2;
        }
    }

    static class EmployeeRepository {
        void save(Employee e) {
            System.out.println("Saving " + e.name + " to database...");
        }
    }
```

```
    static class ReportGenerator {
        void generate(Employee e) {
            System.out.println("Generating report for " + e.name);
        }
    }

    public static void main(String[] args) {

        Employee emp = new Employee("Rahul", 50000);

        SalaryCalculator sc = new SalaryCalculator();
        EmployeeRepository repo = new EmployeeRepository();
        ReportGenerator rg = new ReportGenerator();

        System.out.println(sc.calculate(emp));
        repo.save(emp);
        rg.generate(emp);
    }
}
```

Now,

1. Each class has **one responsibility**
2. Changes in email won't affect reports

2. Open/Closed Principle (OCP)

The Open/Closed Principle states that software entities such as classes, modules, and functions should be open for extension but closed for modification. This means that you should be able to add new functionality without altering existing tested code.

Importance:

- Reduces risk of introducing bugs
- Encourages scalable design
- Supports plug-and-play behavior
- Improves stability
- Makes systems easier to extend

Example Violating OCP:

In this design, discount logic is handled using conditional statements.

```
public class Main {

   static class DiscountCalculator {

      double calculate(String type, double amount) {

         if (type.equals("REGULAR"))
            return amount * 0.1;
```

```
        if (type.equals("PREMIUM"))
            return amount * 0.2;

        return 0;
    }

    public static void main(String[] args) {
        DiscountCalculator dc = new DiscountCalculator();
        System.out.println(dc.calculate("REGULAR", 1000));
    }
  }
}
```

Example Following OCP:

The solution is to use abstraction and polymorphism.

```
public class Main {

  static class DiscountService {

    DiscountStrategy strategy;

    DiscountService(DiscountStrategy strategy) {
        this.strategy = strategy;
    }

    double calculate(double amount) {
        return strategy.apply(amount);
    }
  }
```

```
    public static void main(String[] args) {

        DiscountService regular =
                new DiscountService(new RegularDiscount());

        DiscountService premium =
                new DiscountService(new PremiumDiscount());

        System.out.println(regular.calculate(1000));
        System.out.println(premium.calculate(1000));
    }
}
```

Q2. Discuss in detail about the violations in SRP and OCP along with their fixes.

Ans.

The Single Responsibility Principle (SRP) and Open/Closed Principle (OCP) are fundamental object-oriented design rules from the SOLID framework.

Violations of these principles result in rigid, fragile, and difficult-to-maintain systems.

This section discusses how these principles are violated in practice and how such violations can be corrected.

A: Violations of Single Responsibility Principle (SRP)

1. What is an SRP Violation

SRP is violated when:

- A class performs multiple unrelated tasks
- A class has more than one reason to change
- Business logic, persistence, and presentation logic are mixed
- The class becomes large and tightly coupled
- Such classes are often called “God Classes.”

2. Example of SRP Violation.

The following class handles:

- Salary Calculation
- Report Printing
- Database Operations

```
public class Main {

  static class Employee {
    String name;
    double baseSalary;

    Employee(String name, double baseSalary) {
      this.name = name;
      this.baseSalary = baseSalary;
    }

    double calculateSalary() {
      return baseSalary * 1.25;
    }

    void saveToDatabase() {
      System.out.println("Saving employee to database...");
    }

    void printReport() {
      System.out.println("Printing employee report...");
    }
  }
```

```
    public static void main(String[] args) {
        Employee e = new Employee("Arjun", 40000);
        System.out.println(e.calculateSalary());
        e.saveToDatabase();
        e.printReport();
    }
}
```

This class may change because:

- Salary Calculation Change
- Database technology changes
- Report format changes

Hence the class has three responsibilities, which is a clear violation of SRP

3. Fixing SRP violations

The solution is to separate concerns by moving each responsibility into its own class.

```
public class Main {

    static class Employee {
        String name;
        double baseSalary;

        Employee(String name, double baseSalary) {
            this.name = name;
            this.baseSalary = baseSalary;
        }
    }

    static class SalaryCalculator {
        double calculate(Employee e) {
            return e.baseSalary * 1.25;
        }
    }
```

```
    static class EmployeeRepository {
        void save(Employee e) {
            System.out.println("Saving " + e.name + " to database...");
        }
    }

    static class ReportPrinter {
        void print(Employee e) {
            System.out.println("Printing report for " + e.name);
        }
    }

    public static void main(String[] args) {
        Employee emp = new Employee("Arjun", 40000);

        SalaryCalculator sc = new SalaryCalculator();
        EmployeeRepository repo = new EmployeeRepository();
        ReportPrinter printer = new ReportPrinter();

        System.out.println(sc.calculate(emp));
        repo.save(emp);
        printer.print(emp);
    }
}
```

B: Violations of Open/Closed Principle (OCP)

1. Meaning of OCP Violation

An OCP violation occurs when:

- A class must be edited whenever new behavior is added
- Logic is controlled by large if-else or switch statements
- No abstraction is used
- Concrete implementations are tightly coupled

2. Example of OCP violation

In the following design, discounts are computed using conditional logic.

```
public class Main {

    static class DiscountCalculator {

        double calculate(String type, double amount) {

            if (type.equals("REGULAR"))
                return amount * 0.1;

            if (type.equals("PREMIUM"))
                return amount * 0.2;

            return 0;
        }

        public static void main(String[] args) {
            DiscountCalculator dc = new DiscountCalculator();
            System.out.println(dc.calculate("REGULAR", 1000));
        }
    }
}
```

3. Fixing OCP Violations

Fix for OCP Violations

The solution is:

```
public class Main {

    static class DiscountService {

        DiscountStrategy strategy;

        DiscountService(DiscountStrategy strategy) {
            this.strategy = strategy;
        }
```

```
        double calculate(double amount) {
            return strategy.apply(amount);
        }
    }

    public static void main(String[] args) {

        DiscountService regular =
                new DiscountService(new RegularDiscount());

        DiscountService premium =
                new DiscountService(new PremiumDiscount());

        System.out.println(regular.calculate(1000));
        System.out.println(premium.calculate(1000));
    }
}
```

Q3. Design an HLD for an Online Examination System applying these principles.

Ans. On Draw.io File